

A I I N T E R V I E W A G E N T



PROBEAI

An AI that listens, thinks, adapts and probes deeper.

TEAM

NOVA NIRMAN

PROJECT

PROBE AI

AI PROMPTS LOG · FRONTEND & BACKEND

HACKATHON SUBMISSION DOCUMENT

Table of Contents

PROBE AI · AI PROMPTS LOG

Section A: Frontend Prompts (AI-PROMPTS-FRONTEND.md)

- 1. Connect interview page to the real backend API
- 2. UI/UX changes: remove AI listener panel, reorder input modes, real scoring
- 3. Recover accidentally-deleted UI components
- 4. Feedback page: show real backend data instead of hardcoded scores
- 5. Show all 20 candidates (only 8 were visible)
- 6. Fix conflicting nested scrollbars on candidates page
- 7. Restore broken landing page animations ("AI Mirror" hero section)
- Tools used — Frontend

Section B: Backend Prompts (AI-PROMPTS-BACKEND.md)

- 1. Initial backend build brief (given to Cursor)
- 2. LLM provider migration (Anthropic → Gemini → Groq)
- 3. CORS fix
- 4. Candidate schema fix (422 error)
- 5. Strict answer evaluation + scoring prompt fix
- Tools used — Backend

1. Connect interview page to the real backend API

Connect my frontend's interview page to my real backend API instead of the current mock/simulated data. Here's exactly what to do:

File to edit: `src/routes/interview.$candidateId.tsx`

What's wrong: This file currently fakes the entire interview using `setTimeout` and a local question list. No real network call is made to my backend at all.

What to do:

1. Add a `callInterviewApi` helper that POSTs to `${API_BASE}/api/interview`, where `API_BASE` defaults to `http://localhost:8080` and can be overridden with `VITE_API_BASE_URL`.
2. Generate one stable `sessionId` (`crypto.randomUUID()`) per interview attempt.
3. Call the backend to start the interview once the cinematic entry animation finishes, and store the question text it returns instead of pulling from the local mock question list.
4. On answer submit, POST `{ sessionId, message: answer }`, and either show the next question or – when done: `true` – store the structured feedback object and move to the completion phase.
5. Add an inline error banner if a backend call fails.
6. Remove old `useEffects` that auto-advanced the interview on fixed timers.

Also: create a `.env` with `VITE_API_BASE_URL=http://localhost:8080`, ensure `.env` is gitignored.

2. UI/UX changes: remove AI listener panel, reorder input modes, real scoring

1. Left panel: remove the "AI Listener" panel and replace it with a Question Progress Tracker showing Q1, Q2, Q3 ... up to Q8 (or however many questions the interview runs), listed vertically. A question the candidate has attempted should turn green. A question they skipped should have no color. Only the question numbers need to be shown, not the question text itself.
2. Answer analysis: change the scoring so it actually reflects answer quality – if the candidate gives a wrong answer, the score should go DOWN; if they give a correct answer, the score should go UP. If a question is skipped, it should add 0 to the score.
3. In the answer input section, put the "Type" option first (visible by default) and the "Voice" option second, so the candidate can type their answer directly instead of voice being the default.

(This prompt was given as a description of desired behavior; the actual code changes were implemented iteratively across the interview page component — adding a `questionHistory` state array to track attempted/skipped questions per number, wiring the backend's per-turn `score` field into the Observation Panel metrics, and swapping the default `inputMode` from "voice" to "type" with the Type/Voice toggle buttons reordered accordingly.)

3. Recover accidentally-deleted UI components

My file `src/routes/interview.$candidateId.tsx` is broken. While manually integrating a real backend API, several component functions were accidentally deleted: `HolographicQuestion`, `InteractionZone`, `ObservationPanel`, `AIThinkingOverlay`, `CompletionFlow`.

Recover the original versions (check git history, or the Lovable-connected project) rather than rewriting them from scratch, to preserve the original cinematic/glassmorphism design. Do NOT modify the already-working `InterviewPage` logic (real API calls, `sessionId` handling).

4. Feedback page: show real backend data instead of hardcoded scores

The `DetailedFeedbackPage` component only shows hardcoded/dummy scores (e.g. fixed 88%, 85% per-question scores in a `QUESTION_JOURNEY` array) instead of the real summary, strengths, gaps, and next-step recommendations returned by the backend after an interview completes. Wire the real feedback object (`summary/strengths/gaps/next`) from the backend response into this page's existing cards/sections WITHOUT changing the original visual design, layout, or any hardcoded visual elements that aren't tied to real data (e.g. keep the existing skill constellation visual, just make the `summary/strengths/gaps/next` text pull from real data).

5. Show all 20 candidates (only 8 were visible)

My candidates list page only shows 8 out of 20 candidates. Add a "Show More" button so the remaining 12 become visible and selectable – without changing any existing UI/UX, styling, layout, or design of the page or the candidate cards.

Find where the candidates array is sliced/limited to 8, add a "Show More" button styled to match the existing design system, reveal the remaining 12 using the same card component.

Every candidate must remain fully clickable/selectable via the existing selection flow.

6. Fix conflicting nested scrollbars on candidates page

There are TWO nested scrollbars fighting each other: a small OUTER scrollbar is doing the scrolling, but the INNER container – which holds all 20 candidate cards and is taller than the viewport – is NOT scrolling itself, so I can't reach candidates below the fold.

Fix this so there is exactly ONE scrollable region – remove the redundant/conflicting scroll container so normal page/mouse-wheel scrolling reaches all 20 candidate cards. Do not change any visual design or the "Show More" logic – only fix which container scrolls.

7. Restore broken landing page animations (“AI Mirror” hero section)

On my landing page, the "AI MIRROR" hero section (circular holographic visual with a pulsing center dot, floating particle nodes, a waveform/audio bar animation, and animated percentage rings on the "ANALYZING KEY AREAS" cards) used to be animated but is now completely static.

Find the component (search for "AI MIRROR", "Beyond Answers", "AI INTERVIEWER", or "ANALYZING KEY AREAS"). Check for: Framer Motion elements that lost their animate/transition props; CSS keyframe animation utility classes (animate-hex-breathe, animate-node-float, animate-pulse-ring, animate-waveform-bar, animate-ring-fill, animate-holo-flicker, animate-float-slow, animate-spin-slow) removed from classNames even though the @keyframes still exist in styles.css; or a parent element clipping/resetting the animation. Restore motion without changing visual design, layout, or copy.

Tools used — Frontend

- **Cursor** — initial API integration wiring
- **Claude (claude.ai)** — debugging (CORS, ports, broken/missing components recovery via GitHub), drafting structured prompts for Antigravity
- **Google Gemini** — iterative UI/scoring changes (left panel redesign, input mode reorder, feedback page real-data wiring)
- **Antigravity** — recovering deleted components, candidates list pagination + scroll fix, landing page animation fix

Section B: Backend Prompts (AI-PROMPTS-BACKEND.md)

1. Initial backend build brief (given to Cursor)

Build a FastAPI backend for an "AI Interview Agent" that conducts a personalized, multi-turn technical interview based on a candidate's cohort progress, then returns structured feedback. I'll hook up my own frontend separately, so focus purely on the backend + database.

```
## Files I'll provide in the project folder
- `curriculum.json` – 31-day curriculum, 8 modules, each day has title/type/tools/objectives
- `candidates.json` – 20 candidate profiles, each with a `member` block (name, role, experience, education, status) and a `missions` array (day, title, passed/skipped, attempts) plus `signals` (commitDays, missionsCompleted, missionsFirstTry)

## API contract – must match exactly
POST /api/interview, no auth, state keyed by sessionId.
Start: { "sessionId": "...", "candidate": {...} } → { "reply": "...", "done": false }
Turn: { "sessionId": "...", "message": "..." } → { "reply": "...", "done": false }
End: { "reply": "...", "done": true, "feedback": { "summary", "strengths", "gaps", "next" } }

## Functional requirements
- Minimum 8 questions across at least 4 different curriculum days.
- Follow-up questions generated dynamically from the candidate's previous answer.
- Conversation context maintained across turns.
- Candidate-aware question selection: prioritize skipped/high-attempt days (weak areas), probe depth on first-try passes (strong areas).
- Structured JSON feedback at the end.
- Use an LLM API for interview question generation, follow-ups, and final feedback JSON.

## Database requirements
- SQLite (via SQLAlchemy) instead of pure in-memory storage.
- Persist: sessions, full turn-by-turn transcript, final feedback per session.
- Debug endpoints: GET /api/interview/{sessionId} and GET /api/interview

## Project structure
FastAPI app with: models (Pydantic + SQLAlchemy), db session handling, candidate analyzer (weak/strong day extraction), LLM wrapper, interview orchestration logic, feedback generator.
```

2. LLM provider migration (Anthropic → Gemini → Groq)

Requested directly, working turn-by-turn: convert `app/services/llm.py`, `app/config.py`, `app/main.py`, and `app/routers/interview.py` between LLM providers due to API access/quota issues — first from Anthropic Claude to Google Gemini, then from Gemini to Groq (and back to Groq again after a Gemini regional quota block). Each migration preserved the same prompt logic and JSON response contract, only swapping the SDK client, API key/config field names, and provider-specific error handling (auth errors, rate-limit errors).

3. CORS fix

Add CORS middleware to the FastAPI backend so the frontend (running on a different port, e.g. `localhost:3000`) can call `http://localhost:8080/api/interview` from the browser without being blocked by CORS policy. Use `fastapi.middleware.cors.CORSMiddleware` with `allow_origins=["*"]` for hackathon/demo purposes.

4. Candidate schema fix (422 error)

The backend's Candidate/Member Pydantic model requires a ``status`` field that the frontend's local candidate data doesn't send, causing a 422 Unprocessable Content error. Make ``status`` optional with a default value (e.g. `"IN_PROGRESS"`) instead of a required field, so the frontend's existing candidate payload validates successfully without needing to be modified.

5. Strict answer evaluation + scoring prompt fix

My AI interview backend has ONE specific bug: the LLM is not correctly scoring candidate answers. Fix ONLY this – do not change anything else (no UI changes, no restructuring, no new features).

The bug: Even when given a clearly wrong, nonsensical, or gibberish answer, the LLM still returns a high/positive score instead of a negative one. It is not actually evaluating correctness – it's defaulting to a lenient/positive score regardless of answer quality.

What to do (in `app/services/llm.py`):

- Strengthen the scoring instructions in `build_system_prompt` with an explicit rubric:
+8 to +10 only if technically correct AND complete; +1 to +7 for partial/vague; -1 to -7 for clear technical errors; -8 to -10 for gibberish/completely wrong/off-topic/"I don't know". Do not default to a positive score – the score must be justified by the actual content of the answer. A polite or confident tone does NOT make an answer correct.
- In `decide_next_turn`, `generate_question_for_day`, and `generate_follow_up`, add a reminder

before the JSON response block: "Before choosing a score, first silently judge: is this answer factually/technically correct for the question asked? Base the score strictly on that judgment, not on politeness or effort."

3. Add a `score: int = 0`` field to `InterviewResponse`, `LLMQuestionResult`, and `LLMFollowUpDecision` in `app/models/pydantic_models.py`, and thread the LLM's returned score value through `app/services/interview.py`'s `process_turn` so it's included in the API response sent to the frontend.
4. Do NOT change function signatures, JSON response shape/keys otherwise, imports, or any other file.

Tools used — Backend

- **Cursor** — initial backend scaffolding, LLM provider migrations, CORS/schema/scoring fixes
- **Claude (claude.ai)** — architecture planning, debugging (ports, git/GitHub setup, quota troubleshooting), drafting the scoring-fix prompt